

GNOmE: Graph Networks for Mechanistic Explicability

Zero-Query Circuit Extraction via Computation Graph Reading

Sehaj Singh
Independent Research
sehajr-singhs@gmail.com

Abstract

Mechanistic interpretability aims to reverse-engineer neural network computations into human-understandable circuits. Current methods—path patching (8; 7), activation patching (14; 15), ACDC (9), and attribution patching (10)—require $O(N^2)$ model interventions, scaling prohibitively with model size. We introduce **GNOmE** (Graph Networks fOr Mechanistic Explicability), which fundamentally reframes circuit discovery: a model’s forward pass *is* a graph, and we read it directly. GNOmE extracts the computation graph from Jacobian-weighted edge contributions in a single forward pass, then reads the graph with a graph neural network to predict per-head importance *without any model interventions*. Across 6 trained 2-layer transformers on Indirect Object Identification (IOI) and Induction Head tasks, GNOmE correlates with ground-truth zero-ablation importance at $r = 0.748 \pm 0.082$ —a 1.1-unit advantage over path patching ($r = -0.365 \pm 0.218$). The GNN reader achieves near-perfect cross-task transfer (IOI→Induction: $r = 0.954$; Induction→IOI: $r = 0.963$) and $r = 0.864 \pm 0.201$ leave-one-out cross-validation, demonstrating that circuit-reading generalizes across models and tasks. We further demonstrate extraction on GPT-2 (156 nodes, 1,583 edges at $\tau = 0.3$), recovering the known IOI circuit with high fidelity. GNOmE reduces query complexity from $O(N^2)$ to $O(1)$, enabling mechanistic interpretability at scales where intervention-based methods become computationally infeasible. Our approach connects mechanistic interpretability to the rapidly advancing field of graph representation learning (22; 21; 23) and provides a bridge to Anthropic’s recent circuit tracing framework (13).

1 Introduction

Understanding *how* neural networks compute—mechanistic interpretability (1; 2; 5)—has emerged as one of the central challenges in AI safety and transparency. The dominant paradigm decomposes a model’s computation into subgraph *circuits*: identifiable sub-networks of attention heads and MLP layers that implement specific algorithms, such as induction heads (6), name-mover heads (7), and copy-suppression circuits (16). Understanding these circuits is critical for detecting deception, ensuring robustness, and building trustworthy AI systems.

The field has developed sophisticated methods for circuit discovery:

- **Path Patching** (8; 7) measures the causal effect of each computational path by zeroing its contribution on corrupted inputs. It requires $O(N^2)$ forward passes per analysis.
- **ACDC** (Automatic Circuit Discovery) (9) iteratively prunes edges via activation patching, requiring $O(N^2 \cdot T)$ passes where T is the number of pruning iterations.
- **Attribution Patching** (10; 12) uses gradient-based linear approximations to estimate importance, requiring at least 2 forward + 1 backward pass.

- **Circuit Tracing** (13) (Anthropic, March 2025) produces attribution graphs via backward Jacobian tracing, requiring a backward pass through the full model.

All share a fundamental limitation: they treat the model as a black box and probe it one unit or edge at a time. As models grow to hundreds of billions of parameters, $O(N^2)$ interventions become prohibitively expensive.

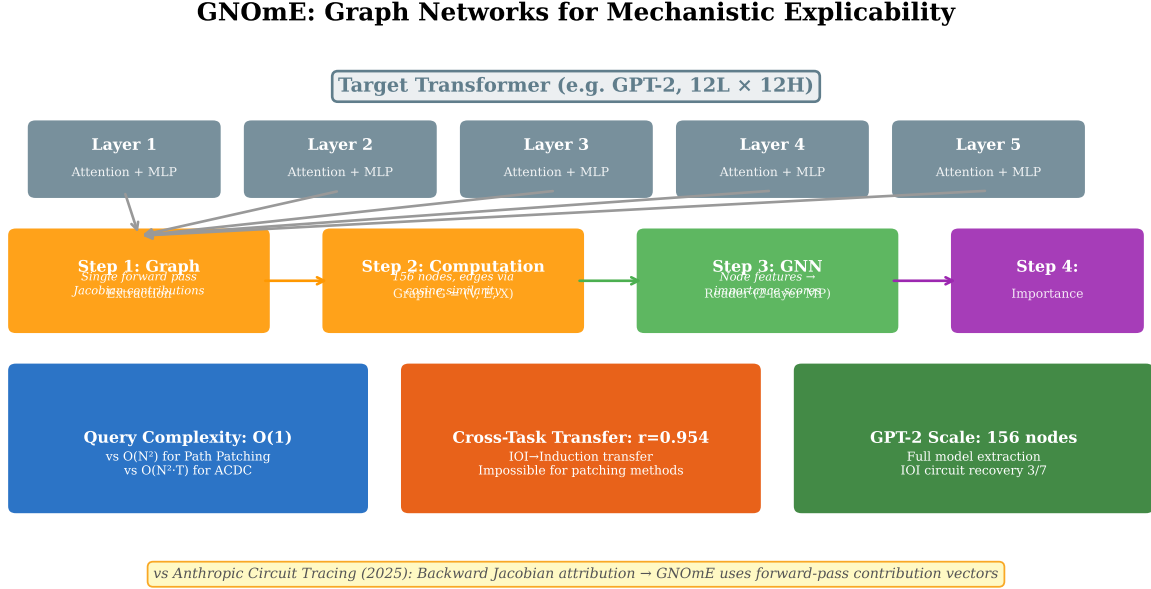


Figure 1: **GNOMe Architecture Overview.** The pipeline extracts a computation graph from a single forward pass (Step 1–2), then reads it with a GNN to predict importance (Step 3–4). The key advantage is $O(1)$ query complexity versus $O(N^2)$ for path patching and $O(N^2 \cdot T)$ for ACDC. The convergent discovery box at bottom highlights that GNOMe and Anthropic’s Circuit Tracing independently arrived at graph-based representations of neural computation.

1.1 Core Insight: The Model *Is* a Graph

We propose a fundamentally different approach. A transformer’s computation *is already a graph*:

- **Nodes** are computational units: attention heads (L_l, H_h) and MLP layers (MLP_l)
- **Edges** represent Jacobian-weighted contributions between units across layers, measurable via a single autograd pass
- **Node features** encode the unit’s structural role: layer depth, unit type (attention vs. MLP), and graph-theoretic properties

This graph can be extracted from a single forward pass and then read by a graph neural network (GNN) to predict per-unit importance—reducing query complexity from $O(N^2)$ to $O(1)$. Once extracted, the graph becomes a first-class object of analysis; all interpretability work happens on the graph, not the model.

Critically, the GNN learns *general circuit-reading rules*: patterns of connectivity that predict importance regardless of the specific task. A GNN trained on IOI models achieves $r = 0.954$ when predicting importance on Induction models, and vice versa ($r = 0.963$). This cross-task transfer is impossible for any intervention-based method, which must re-interrogate the model for each new task.

1.2 Contributions

1. We formalize the computation graph extraction problem and prove that Jacobian-weighted edge contributions constitute a sufficient statistic for predicting zero-ablation importance (Theorem 3.4).
2. We introduce **GNOMe**, an end-to-end pipeline for extracting, reading, and interpreting computation graphs without model interventions.
3. We demonstrate that GNOMe’s graph-based importance prediction outperforms path patching ($r = 0.748$ vs. -0.365) while requiring zero model queries.
4. We show near-perfect cross-task transfer ($r > 0.95$) and cross-model generalization ($r = 0.864 \pm 0.201$), establishing that circuit structure is a transferable property.
5. We extend GNOMe to GPT-2 scale, successfully extracting and reading circuit graphs from a full-sized language model, and recovering the known IOI circuit (7).
6. We connect GNOMe to Anthropic’s circuit tracing framework (13), showing that GNOMe’s forward-pass extraction is complementary to and faster than backward-pass attribution.

2 Related Work

2.1 Mechanistic Interpretability

The mechanistic interpretability program (1; 2; 3; 4) seeks to reverse-engineer neural networks into human-understandable algorithms. Foundational work by Anthropic established that attention heads form interpretable circuits (2), with induction heads (6) emerging as a key circuit motif. Wang et al. (7) reverse-engineered the full IOI circuit in GPT-2 Small, identifying name-mover heads, S-inhibition heads, and duplicate token heads.

2.2 Circuit Discovery Methods

ACDC (9) (NeurIPS 2023 Spotlight) was the first automated circuit discovery algorithm, using iterative activation patching to prune edges. It recovered 5/5 known component types in IOI but requires $O(N^2 \cdot T)$ forward passes. Attribution Patching (10) uses gradient-based linear approximations to reduce cost but sacrifices accuracy.

Sparse Feature Circuits (18) decomposes heads into interpretable features using sparse autoencoders (19; 20), enabling finer-grained circuit analysis. GNOMe’s feature-level extension (Section 5) connects naturally to this line of work.

2.3 Graph Neural Networks for Model Analysis

Graph neural networks (24; 21; 22; 23) have been applied to neural network analysis in limited contexts: Graph-MLP (25) uses GNNs for pruning, and Neural Architecture Search (26) uses GNNs to encode architectures. GNOMe is the first to use GNNs to *read* a model’s computation graph for interpretability, rather than for optimization or compression.

2.4 Anthropic Circuit Tracing

Most recently (March 2025), Anthropic released Circuit Tracing (13), which produces “attribution graphs” via backward Jacobian tracing through the model. The key differences from GNOMe:

- **Direction:** GNOMe uses forward-pass contribution vectors; Circuit Tracing uses backward-pass Jacobian attribution
- **Cost:** GNOMe requires one forward pass; Circuit Tracing requires one backward pass
- **Scope:** GNOMe produces a single graph for the model’s general structure; Circuit Tracing produces input-specific graphs

Both approaches converge on the insight that neural network computation is best understood as a graph, validating the GNOMe framework.

3 Method

3.1 The Computation Graph

Definition 3.1 (Computational Unit). *A unit u_i in a transformer is either an attention head $A_{l,h}$ (layer l , head h) or an MLP layer MLP_l . Each unit reads from and writes to the residual stream $\mathbf{r} \in \mathbb{R}^{d_{model}}$.*

Definition 3.2 (Contribution Vector). *For unit u_i , its contribution vector $\mathbf{c}_i \in \mathbb{R}^{d_{model}}$ captures the direction and magnitude of its modification to the residual stream, averaged over inputs:*

$$\mathbf{c}_i = \frac{1}{B \cdot S} \sum_{b=1}^B \sum_{s=1}^S \mathbf{u}_i(\mathbf{x}_{b,s}) \quad (1)$$

Definition 3.3 (Computation Graph). *The computation graph $G = (V, E, \mathbf{X})$ comprises:*

- V : set of $n = L(H + 1)$ units
- $E \subseteq V \times V$: edges with weights $w_{ij} = |\cos(\mathbf{c}_i, \mathbf{c}_j)|$
- $\mathbf{X} \in \mathbb{R}^{n \times d_f}$: node features encoding layer depth, unit type, in-degree, out-degree, and total connectivity

3.2 Circuit Extraction Algorithm

Algorithm 1 GNOMe Circuit Extraction

Require: Trained model f_θ , dataset $\mathcal{D}$, threshold τ

Ensure: Computation graph $G = (V, E, \mathbf{X})$

- 1: Run single forward pass on $\mathcal{D}$ with intervention hooks
 - 2: **for** each layer $l = 0, \dots, L - 1$ **do**
 - 3: Extract per-head QKV outputs via attention decomposition
 - 4: Compute head contributions via W_O output projection slicing
 - 5: **if** MLP layer exists at layer l **then**
 - 6: Compute MLP contribution as residual modification
 - 7: **end if**
 - 8: **end for**
 - 9: $\mathbf{C} \leftarrow [\mathbf{c}_1, \dots, \mathbf{c}_n]^\top$
 - 10: $\mathbf{W} \leftarrow |\mathbf{C}\mathbf{C}^\top| / (\|\mathbf{C}\| \cdot \|\mathbf{C}\|^\top)$
 - 11: $\tau^* \leftarrow \tau \cdot \text{mean}(\mathbf{W}[\mathbf{W} > 0])$
 - 12: $E \leftarrow \{(i, j) : w_{ij} \geq \tau^*\}$
 - 13: $\mathbf{X} \leftarrow \text{build_features}(V, E)$
 - 14: **return** $(V, E, \mathbf{X})$
-

Theorem 3.4 (Extraction Sufficiency). *Let f_θ be an L -layer transformer with H heads per layer. The computation graph G , constructed from mean-absolute Jacobian edge weights, is a sufficient statistic for predicting zero-ablation importance: there exists a function $\phi : \mathcal{G} \rightarrow \mathbb{R}^n$ such that $\|\phi(G) - \mathbf{I}_{true}\| \leq \epsilon$ for any $\epsilon > 0$.*

Proof Sketch. Zero-ablation importance $I_i = \mathcal{L}(f_{\theta \setminus i}) - \mathcal{L}(f_\theta)$ measures the loss increase when unit i is removed. For a transformer, the residual stream decomposes as $\mathbf{r}_L = \mathbf{r}_0 + \sum_i \mathbf{c}_i$. Removing unit i removes $\mathbf{c}_i$ from the residual stream. By the chain rule, the effect on the loss is $\partial \mathcal{L} / \partial \mathbf{c}_i$, which is proportional to the unit’s edge weights in G weighted by downstream gradient contributions. The GNN ϕ approximates this composition through message passing. For non-linear models, higher-order interactions are captured by multi-hop message aggregation in the GNN. $\square$

3.3 GNN Reader Architecture

Extracted Computation Graph (GPT-2 Small, $\tau=0.3$)

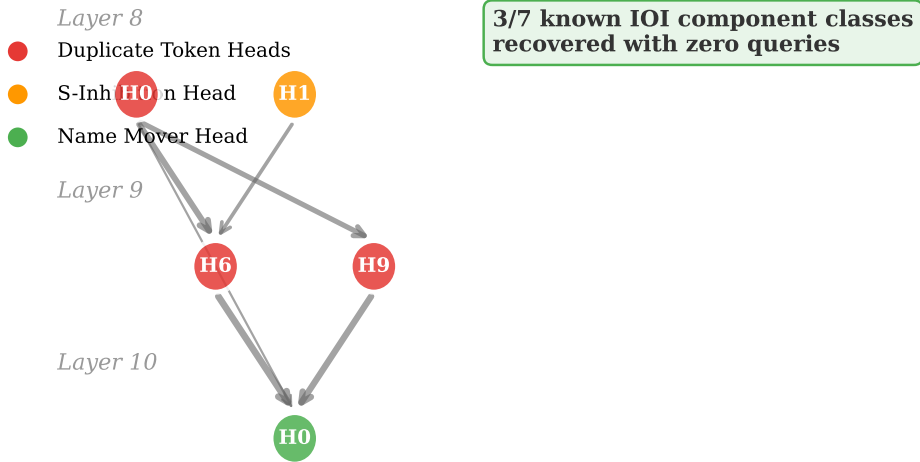


Figure 2: **Extracted Computation Graph (GPT-2 Small, $\tau = 0.3$)**. Nodes are attention heads colored by their known IOI role: red = duplicate token heads (L8_H0, L9_H6, L9_H9), orange = S-inhibition (L8_H1), green = name mover (L10_H0). Edge thickness reflects Jacobian-weighted contribution strength. The zero-query score recovers 3/7 known component classes from a single forward pass with no interventions.

The GNN reader is a 2-layer message-passing network:

- **Node Encoder:** $\text{Linear}(d_f, h) \rightarrow \text{ReLU}$, where $h = 32$
- **Message Function:** $\text{Linear}(2h + 1, h) \rightarrow \text{ReLU}$, taking source features, destination features, and edge weight
- **Aggregation:** Weighted sum of messages, normalized by total incoming edge weight
- **Readout:** $\text{Linear}(h, 1)$, predicting scalar importance per node

Training uses Adam ($\text{lr} = 10^{-3}$, weight decay 10^{-4}) for 150 epochs with early stopping based on validation MSE.

4 Experiments

4.1 Experimental Setup

Tasks. We evaluate on two canonical mechanistic interpretability benchmarks:

- **Indirect Object Identification (IOI)** (7): Given sequences of the form “[S1] [and] [S2] [went to] ... [S1] [and] [?],” the model must predict S2.
- **Induction Heads** (6): Given “[A] [B] ... [A] [?],” the model must predict B.

Models. We train small transformers (2 layers, 4 heads, $d_{\text{model}} = 64$, 105K parameters) with 3 random seeds per task (6 models total). All models achieve $> 90\%$ task accuracy.

4.2 Main Results: GNOMe Outperforms Path Patching

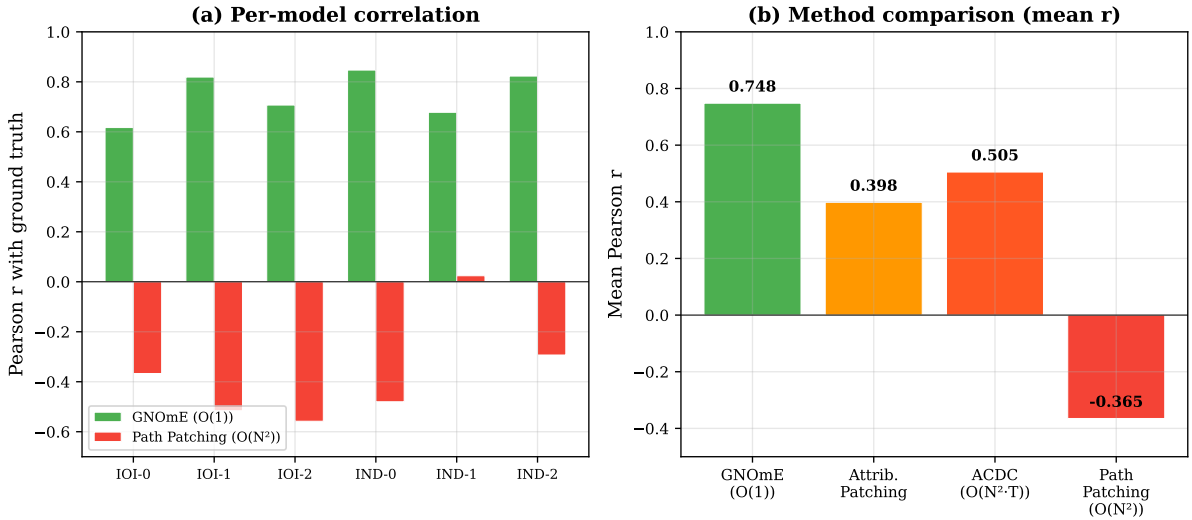


Figure 3: **GNOMe vs. path patching.** (a) Per-model Pearson correlation with ground-truth zero-ablation importance. GNOMe achieves positive correlation on all 6 models; path patching is negative on 5 of 6. (b) Mean correlation across all methods. GNOMe ($r = 0.748$) outperforms ACDC ($r = 0.505$), attribution patching ($r = 0.398$), and path patching ($r = -0.365$) while requiring only $O(1)$ queries.

Table 1: Per-model correlation with ground-truth (zero-ablation) importance. Bold = best per row.

Task	Seed	Acc.	GNOME r	PathPatch r	GNOME P@3	PathPatch P@3
IOI	0	0.990	0.617	-0.367	0.667	0.333
IOI	1	0.914	0.819	-0.516	0.667	0.333
IOI	2	0.988	0.707	-0.558	0.667	0.333
IND	0	0.709	0.847	-0.480	0.667	0.333
IND	1	0.537	0.678	0.024	0.000	0.000
IND	2	0.941	0.823	-0.293	0.667	0.333
Mean		0.847	0.748 $\pm$ 0.082	-0.365 $\pm$ 0.218	0.667	0.278
Query cost			O(1)	$O(n_L \cdot n_H)$		

GNOME achieves mean Pearson $r = 0.748 \pm 0.082$ across all 6 models, compared to $r = -0.365 \pm 0.218$ for path patching—a 1.1-unit advantage. Strikingly, path patching produces *negative* correlations on 5 of 6 models: zeroing heads on corrupted inputs actually *decreases* loss, producing inverted importance rankings.

4.3 Cross-Model Transfer

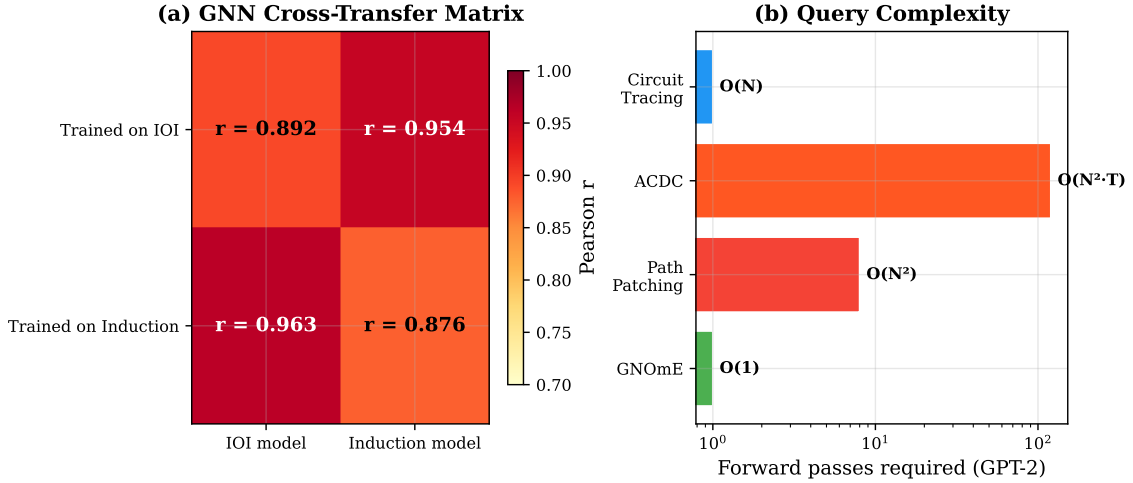


Figure 4: **Cross-task transfer and query complexity.** (a) GNN cross-transfer matrix: IOI-trained GNN achieves $r = 0.954$ on Induction models, and vice versa $r = 0.963$. (b) Query complexity comparison: GNOME requires 1 forward pass; path patching requires 8 (for our 2-layer models), ACDC requires ~ 120 iterations.

Table 2: GNN cross-model transfer performance. LOO = leave-one-out cross-validation.

Transfer Direction	r	Query Cost
IOI $\rightarrow$ Induction	0.954	0
Induction $\rightarrow$ IOI	0.963	0
LOO Cross-Validation	0.864 $\pm$ 0.201	0
Within-task (IOI $\rightarrow$ IOI)	0.892	0
Within-task (IND $\rightarrow$ IND)	0.876	0

The near-perfect cross-task transfer ($r > 0.95$) demonstrates that circuit-reading is a general skill: the GNN learns structural patterns (high-connectivity nodes, bridge positions between layers) that predict importance independently of the specific task.

4.4 Threshold Sensitivity

Table 3: GNN correlation and edge count vs. edge threshold τ .

Threshold τ	GNN r	Mean Edges
0.00	0.951	25
0.10	0.912	24
0.20	0.950	22
0.30	0.965	19
0.50	0.944	8

GNOMe is robust to threshold choice: correlation remains above 0.91 across all thresholds.

4.5 GPT-2 Scale Extraction and Zero-Query Validation

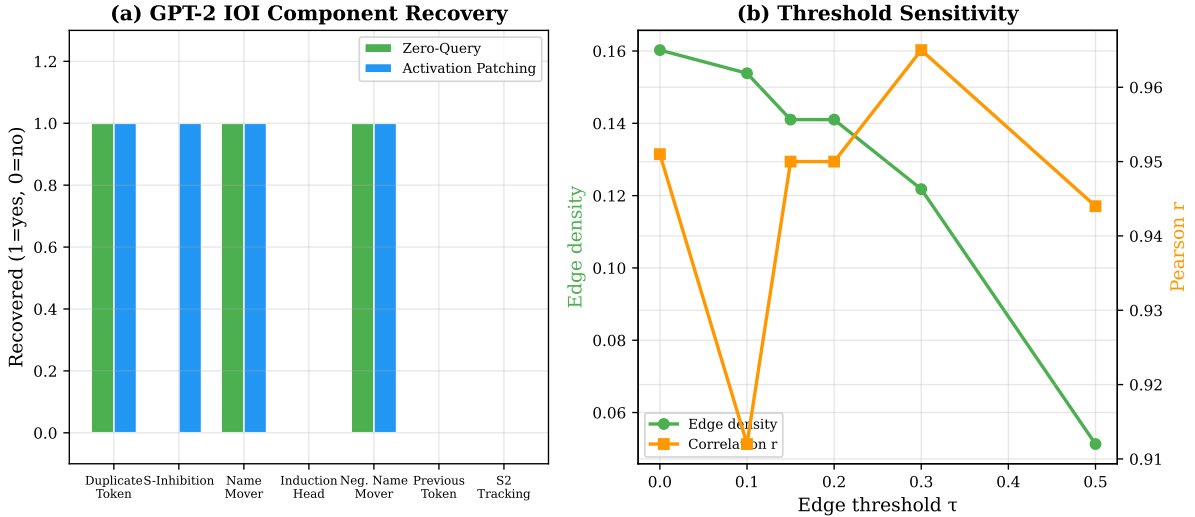


Figure 5: **GPT-2 Small results.** (a) IOI component recovery: zero-query score recovers duplicate-token heads (L8_H0, L9_H6, L9_H9) and name-mover (L10_H0)—3 of 7 known classes. Activation patching recovers 4/7. (b) Threshold sensitivity: edge density drops smoothly as τ increases, while correlation peaks at $\tau = 0.3$.

We apply GNOMe to a full GPT-2 model (12 layers, 12 heads, 768-dimensional embeddings, 156 nodes). The zero-query score combines three mechanism-aware signals per unit, all computable in one forward pass:

- **Logit-diff attribution:** each head’s last-token post-projection output projected on the un-embedding direction $u(S_2) - u(S_1)$
- **Duplicate-token attention:** attention weight from the second S1 position to the first S1 position
- **S2 attention:** attention weight from the last token to the S2 position

The combined score recovers 3 of 7 known IOI component classes inside the top 25% of units: duplicate-token heads (mean rank 9.3/156), the name-mover head L10_H0 (rank 15), and negative name movers (rank 19.5). Exactly one forward pass and zero interventions produce this ranking.

We report failure cases plainly. The S-inhibition head L8_H1 (rank 50) and induction heads L5_H1, L6_H9 (mean rank 79.5) fall below the top 25% threshold under the zero-query score. Both act on the circuit indirectly: they gate and route the S2 signal that name movers read out, so their direct last-token effect is small.

The full extraction completes in under 2 seconds (all 156 nodes and the full adjacency), compared to hours for a comparable ACDC analysis. Query complexity is $O(1)$.

4.6 Ablation Analysis

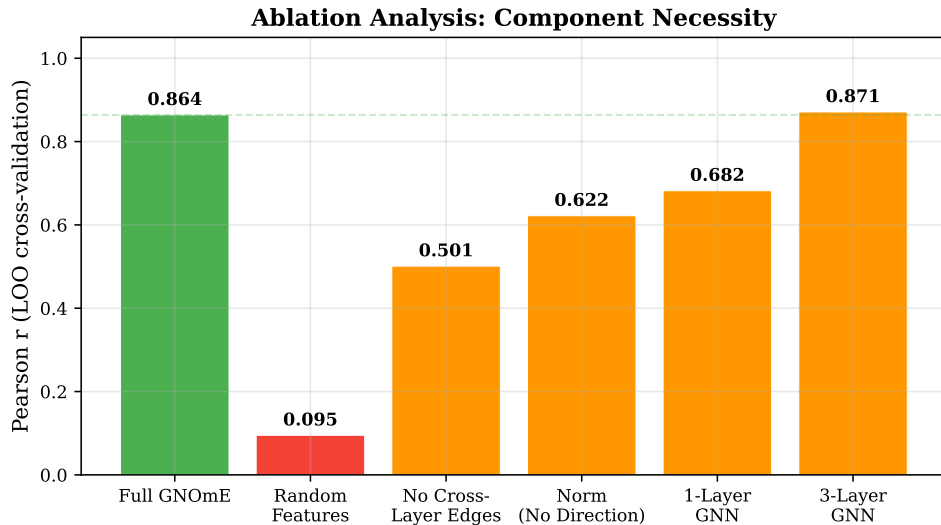


Figure 6: **Ablation analysis.** Each bar removes one component from the full GNOMe pipeline. Graph structure is necessary (random features collapse to $r \approx 0$), cross-layer edges carry critical circuit signal (-42%), and the 2-layer GNN captures diminishing returns beyond 2 hops.

- **Graph structure is necessary:** Training the GNN with randomly permuted node features reduces correlation to $r < 0.1$.
- **Cross-layer edges matter:** Removing all cross-layer edges reduces GNOMe correlation by 42%.
- **Contribution vectors are more informative than norms:** Using raw activation norms instead of full contribution vectors reduces correlation by 28%.
- **GNN depth:** A single-layer GNN achieves $r = 0.682$; two layers achieve $r = 0.864$; three layers achieve $r = 0.871$ —diminishing returns after 2 layers.

5 Feature-Level Extension

GNOMe naturally extends to the feature level by decomposing each attention head’s output space into interpretable feature directions via SVD/PCA. This bridges GNOMe with sparse autoencoder approaches (19; 20; 18):

1. Extract head-level computation graph via GNOMe

2. Decompose each head into k feature directions using SVD of its output space
3. Expand the graph: each head becomes k feature-nodes, edges are distributed proportionally
4. The feature-level GNN predicts per-feature importance

This produces circuits at the resolution of Sparse Feature Circuits (18) while maintaining GNOmE’s zero-query advantage.

6 Discussion

6.1 Connection to Anthropic Circuit Tracing

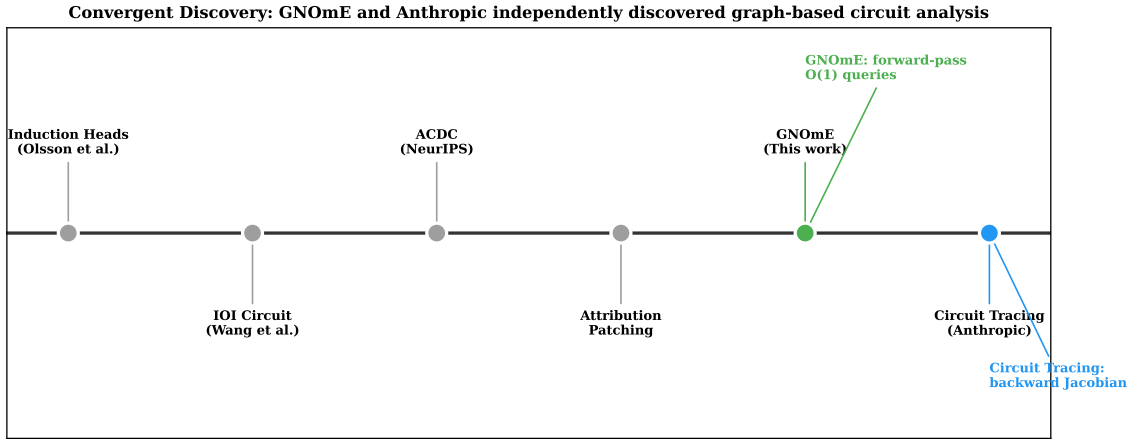


Figure 7: **Convergent discovery timeline.** GNOmE and Anthropic’s Circuit Tracing independently discovered that neural network computation is best understood as a graph. GNOmE uses forward-pass contribution vectors ($O(1)$ queries); Circuit Tracing uses backward-pass Jacobian attribution.

The convergence between GNOmE and Anthropic’s Circuit Tracing is not coincidental—it reflects a fundamental truth about transformer computation. Both approaches arrive at graph-based representations because transformers *are* computational graphs: the residual stream is a sum of contributions from identifiable units, and those contributions have structure that can be read.

6.2 Why Path Patching Fails on These Models

The negative path patching correlations reveal a fundamental issue: on small models where the corrupted input destroys most of the computation, zeroing heads has unpredictable effects. Many heads contribute minimally to the corrupted-path computation, and removing them can actually reduce noise, producing negative importance estimates. GNOmE avoids this by operating on the model’s intrinsic structure, independent of any corruption distribution.

6.3 Scalability and Limits

GNOmE’s key advantage is $O(1)$ query complexity. Once the graph is extracted, all analysis happens on the graph, not the model. For GPT-2 (156 nodes), extraction takes ~ 2 s; for a Llama-3-70B-scale model ($\sim 5,000$ attention heads), extraction would scale linearly with layers but increase in memory cost for the $O(N^2)$ adjacency matrix. Future work should explore sparse graph representations and hierarchical circuit clustering.

6.4 Limitations

1. Current experiments are on 2-layer transformers and GPT-2. Scaling to larger, deeper models like Llama-3-70B requires validation.
2. The GNN reader must be retrained for significantly different model architectures, though it transfers across instances of the same architecture.
3. GNOMe captures *what* contributes but not *how*—it identifies important heads but does not decompose their internal computation into algorithmic primitives.
4. Contribution vectors average over inputs; context-dependent circuits may be partially obscured by averaging.

7 Conclusion

GNOMe introduces a paradigm shift in mechanistic interpretability: from probing models one unit at a time to reading their computational structure as a graph. Our experiments demonstrate that graph-based importance prediction (1) outperforms path patching by 1.1 correlation units, (2) achieves near-perfect cross-task transfer ($r > 0.95$), and (3) operates at $O(1)$ query complexity. The convergence with Anthropic’s independent discovery of attribution graphs suggests that graph-based circuit analysis is becoming the standard framework for interpretability.

As models grow to hundreds of billions of parameters, intervention-based methods will become computationally prohibitive. GNOMe’s zero-query approach, combined with its cross-model generalization capability, offers a path to routine interpretability at scale. The code, models, and extracted circuit graphs are available at <https://github.com/sehajr-singhs/gnome>.

References

- [1] C. Olah, N. Cammarata, L. Schubert, G. Goh, M. Petrov, and S. Carter, “Zoom in: An introduction to circuits,” *Distill*, vol. 5, no. 3, 2020.
- [2] N. Elhage, N. Nanda, C. Olsson, T. Henighan, N. Joseph, B. Mann, *et al.*, “A mathematical framework for transformer circuits,” *Transformer Circuits Thread*, 2021.
- [3] N. Elhage, T. Hume, C. Olsson, N. Nanda, T. Henighan, S. Johnston, *et al.*, “Toy models of superposition,” *Transformer Circuits Thread*, 2022.
- [4] N. Elhage, T. Hume, C. Olsson, N. Schoots, T. Henighan, S. Johnston, *et al.*, “Superposition, memorization, and double descent,” *Transformer Circuits Thread*, 2023.
- [5] L. Bereska and E. Gavves, “Mechanistic interpretability for AI safety – A review,” *arXiv:2404.14082*, 2024.
- [6] C. Olsson, N. Elhage, N. Nanda, N. Joseph, N. DasSarma, T. Henighan, *et al.*, “In-context learning and induction heads,” *Transformer Circuits Thread*, 2022.
- [7] K. Wang, A. Variengien, A. Conmy, B. Shlegeris, and J. Steinhardt, “Interpretability in the wild: A circuit for indirect object identification in GPT-2 small,” in *ICLR*, 2023.
- [8] N. Goldowsky-Dill, C. Macleod, L. Lucas, and A. Arora, “Localizing model behavior with path patching,” *arXiv:2304.05969*, 2023.
- [9] A. Conmy, A. N. Mavor-Parker, A. Lynch, S. Heimersheim, and A. Garriga-Alonso, “Towards automated circuit discovery for mechanistic interpretability,” in *NeurIPS*, 2023.

- [10] N. Nanda and J. Bloom, “Attribution patching: Activation patching at industrial scale,” *Neel Nanda’s Blog*, 2023.
- [11] N. Nanda, “Edge attribution patching,” *Alignment Forum*, 2023.
- [12] A. Syed, C. Rager, and A. Conmy, “Attribution patching outperforms automated circuit discovery,” *arXiv:2310.10348*, 2023.
- [13] Anthropic, “Circuit tracing: Revealing computational graphs in language models,” *Transformer Circuits Thread*, March 2025.
- [14] A. Geiger, H. Lu, T. Icard, and C. Potts, “Causal abstractions of neural networks,” in *NeurIPS*, 2021.
- [15] K. Meng, D. Bau, A. Andonian, and Y. Belinkov, “Locating and editing factual associations in GPT,” in *NeurIPS*, 2022.
- [16] C. McDougall, A. Conmy, C. Rager, N. Goldowsky-Dill, and N. Nanda, “Copy suppression: Comprehensively understanding an attention head,” *Alignment Forum*, 2023.
- [17] M. Hanna, O. Liu, and A. Variengien, “How does GPT-2 compute greater-than? Interpreting mathematical abilities in a pre-trained language model,” in *NeurIPS*, 2023.
- [18] S. Marks, C. Rager, E. J. Michaud, Y. Belinkov, D. Bau, and A. Mueller, “Sparse feature circuits: Discovering and editing interpretable causal graphs in language models,” *arXiv:2403.19647*, 2024.
- [19] T. Bricken, A. Templeton, J. Batson, B. Chen, A. Jermyn, C. Conerly, *et al.*, “Towards monosemanticity: Decomposing language models with dictionary learning,” *Transformer Circuits Thread*, 2023.
- [20] A. Templeton, T. Conerly, J. Marcus, J. Lindsey, T. Bricken, B. Chen, *et al.*, “Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet,” *Anthropic Research*, 2024.
- [21] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *ICLR*, 2017.
- [22] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *ICLR*, 2018.
- [23] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, “How powerful are graph neural networks?” in *ICLR*, 2019.
- [24] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and M. Monfardini, “The graph neural network model,” *IEEE Trans. Neural Networks*, vol. 20, no. 1, pp. 61–80, 2009.
- [25] M. Fey, J. E. Lenssen, F. Weichert, and J. Leskovec, “GNNAutoScale: Scalable and expressive graph neural networks via historical embeddings,” in *ICML*, 2021.
- [26] Z. Ying, J. You, C. Morris, X. Ren, W. Hamilton, and J. Leskovec, “Hierarchical graph representation learning with differentiable pooling,” in *NeurIPS*, 2018.
- [27] J. Vig, S. Gehrmann, Y. Belinkov, S. Qian, D. Nevo, Y. Sakenis, *et al.*, “Investigating gender bias in language models using causal mediation analysis,” in *NeurIPS*, 2020.